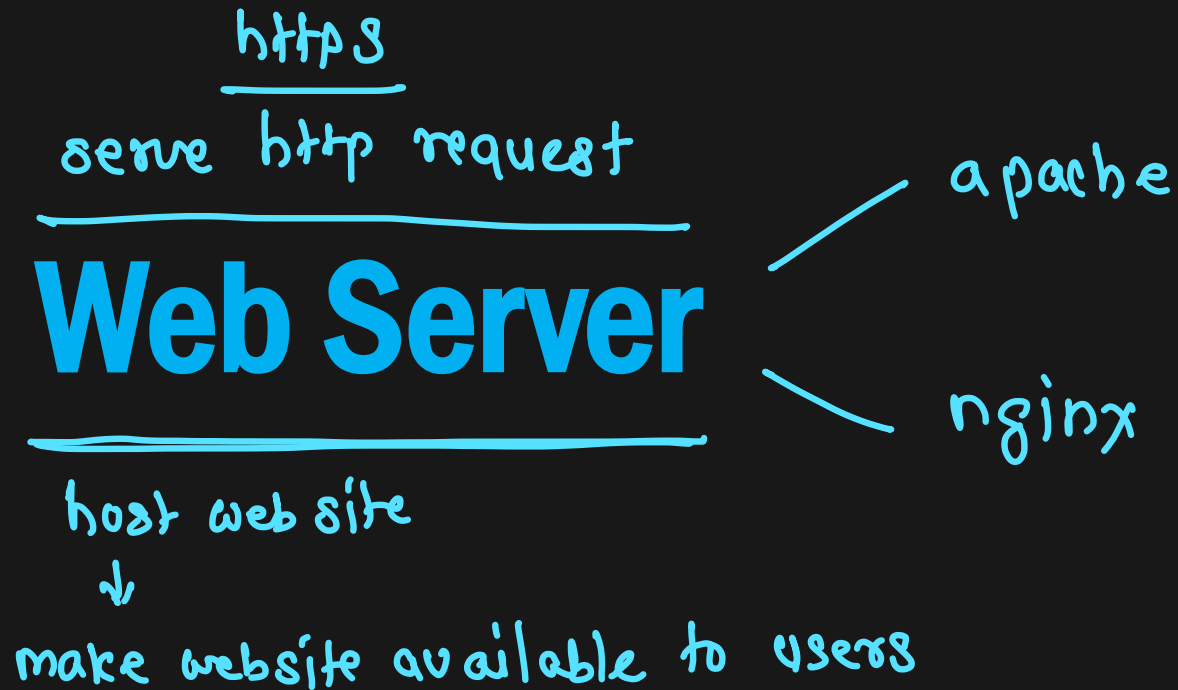




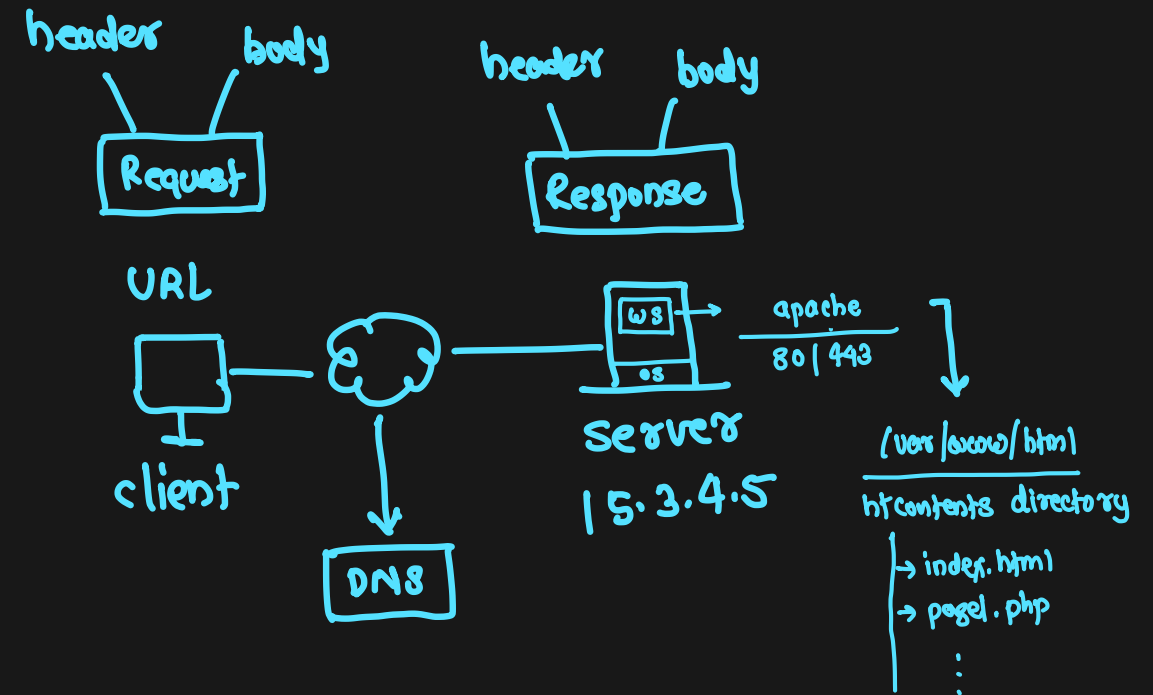
Web Server

→ huge infrastructure → CPU | RAM | storage



- A web server is software (and sometimes hardware) that:
 - Accepts HTTP/HTTPS requests from clients (browsers, apps)
 - Processes the request
 - Returns a response (HTML, JSON, files, etc.)
- Web server flow
 - User enters URL
 - DNS resolves domain → IP
 - Browser sends HTTP request
 - Web server receives request
 - Server processes (static/dynamic)
 - Response sent back

Web Architecture



Understanding HTTP

http uses client-server architecture & request-response pattern



- HTTP traffic makes up a significant portion of the world's Internet traffic, and Apache is a server implementation of HTTP
- Applications such as Firefox, Chrome, Opera, Curl, wget, Edge, Safari, and Internet Explorer are client implementations of HTTP
- As of this writing, HTTP is at version 2, but HTTP version 1.1 is still by far the most widely used version of the protocol
- HTTP/1.1 is documented in RFCs 7230 through 7235
- **Request**
 - When a web client connects to a web server, the client's default method of making this connection is to contact the server's port 80
 - Once connected, the web server says nothing; it's up to the client to issue HTTP-compliant commands (also called verbs or methods) for its requests to the server
 - Along with each command comes a request header that includes information about the client
 - GET / HTTP/1.1
 - Connection: Keep-Alive
 - User-Agent: Mozilla/* (X11;-Linux x86_64;rv:*) Gecko/* Firefox/*
 - Host: localhost:80
 - Accept: text/xml, image/gif, image/jpeg, image/png...
 - Accept-Encoding: gzip, deflate
 - Accept-Language: en-us
 - Accept-Charset: iso-8859-1,*,utf-8

http methods

GET

POST

PUT

PATCH

DELETE

Understanding HTTP

port → number assigned to server process to accept incoming request
→ Standard → 1-1024 → reserved, 3306, 27017
→ Ephemeral → >1024 → assigned randomly by OS

■ Response

- When it receives a blank line, the server knows a request header is complete
- Once the request header is received, it responds with the actual requested content, prefixed by a server header
- The server header provides the client with information about the server, the amount of data the client is about to receive, the type of data coming in, and other information

- HTTP/1.1 200 OK
- Date: Mon, 14 Nov 2022 14:03:31 GMT
- Server: Apache/2.4.* (Fedora)
- Last-Modified: : Mon, 14 Nov 2022 11:41:32 GMT
- ETag: "7888-1f-b80bf300"
- Accept-Ranges: bytes
- Content-Length: 31
- Connection: close
- Content-Type: text/html; charset=UTF-8

Status code

1xx → informational

2xx → success

3xx → redirection

4xx → client error

5xx → server error

■ Ports

- The default port for HTTP requests is port 80, but you can also configure a web server to use a different (arbitrarily chosen) port that is not in use by another service
- This is one of the mechanisms for running multiple web servers or sites on the same host, with each server or site on a different port



Types of Web Servers

■ Static Web Server

- Serves static files (HTML, CSS, JS)
- Fast and lightweight

Example : apache with html contents

■ Dynamic Web Server

- Executes server-side code
- Generates dynamic content
- E.g. PHP, Node, Python (Flask/Django)



Popular web servers

✓ Apache HTTP Server

- Oldest and widely used
- Modular architecture

RH → httpd
Debian → apache2
→ plugins/module

✓ Nginx

- High performance
- Event-driven
- Reverse proxy & load balancer

✓ Microsoft IIS

- Windows-based
- Integrated with .NET

▪ LiteSpeed Web Server

- High-performance alternative to Apache



HTTPD (Apache)

Apache



- Apache is a widely popular open source Hypertext Transfer Protocol (HTTP) server software
- Apache continues to dominate the web server market in terms of use and deployments
- Apache offers some of the following benefits and advantages:
 - It is stable, flexible, and secure → php | perl | Java
 - It is used, backed, and supported by several major sites and organizations
 - The entire program and related components are open source
- It works on most variants of Linux/UNIX and Microsoft Windows

Stack → collection of software used host web application

- platform
- database
- user interface framework (library)
- web server

① AMP → Apache + MySQL + Perl | Python | PHP
↳ Linux (LAMP), MacOS (MAMP),
Windows (WAMP)

- ② MERN → Mongo + Express + React + NodeJS
- ② MEAN → Mongo + Express + Angular + —
- ③ WISA → Win + IIS + SQL Server + Asp.net

Apache administration

■ Install apache

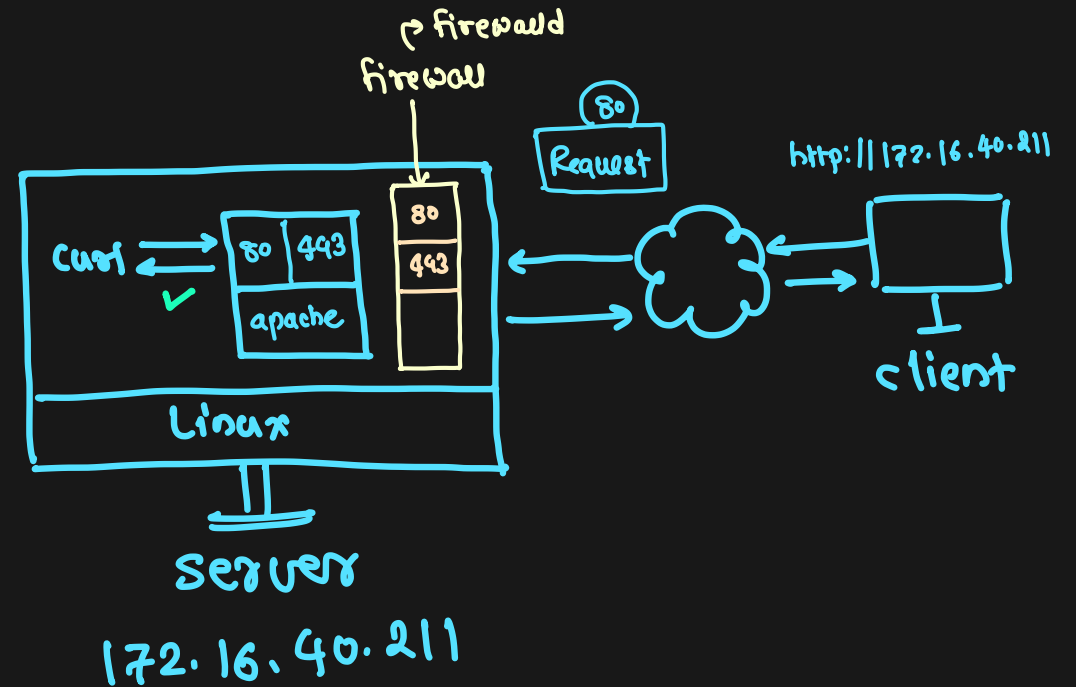
- `sudo dnf update -y`
- `sudo dnf install httpd -y`

■ Managing Apache Service

- `sudo systemctl start httpd`
- `sudo systemctl stop httpd`
- `sudo systemctl restart httpd`
- `sudo systemctl enable httpd`
- `sudo systemctl status httpd`

■ Firewall Configuration

- `sudo firewall-cmd --permanent --add-service=http` = 80
- `sudo firewall-cmd --permanent --add-service=https` = 443
- `sudo firewall-cmd --reload` → refresh



Directory Structure

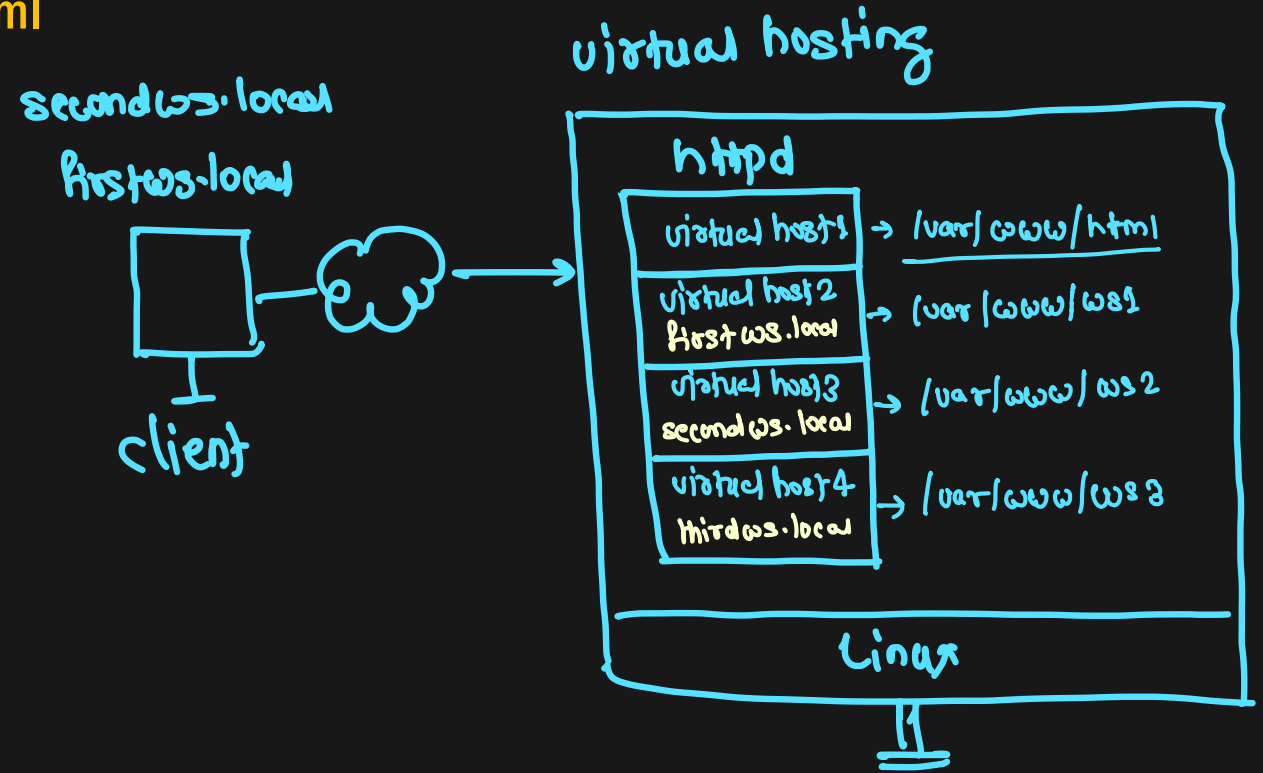


- Main config: /etc/httpd/conf/httpd.conf
- Virtual hosts: /etc/httpd/conf.d/
- Document root: /var/www/html/
- Logs: /var/log/httpd/

Default website hosting

- The default document directory is located at `/var/www/html`
- The default web page name is `index.html`
- The default port configured is `80`

① website 1 → machine 1 → apache
② website 2 → machine 2 → apache
③ website 3 → machine 3 → apache



Virtual Hosting



- Create a configuration

- `sudo nano /etc/httpd/conf.d/example.conf`

```
<VirtualHost *:80>
    ServerName example.local
    DocumentRoot /var/www/mywebsite

    <Directory /var/www/mywebsite>
        AllowOverride All
        Require all granted
    </Directory>
</VirtualHost>
```

Server alias (pointing to `example.local`)

- Add SELinux settings for the directory

- `sudo chcon -R -t httpd_sys_content_t /var/www/example`

- Add SELinux settings for custom port (if required)

- `sudo semanage port -a -t http_port_t -p tcp 8080`



Load Balancing

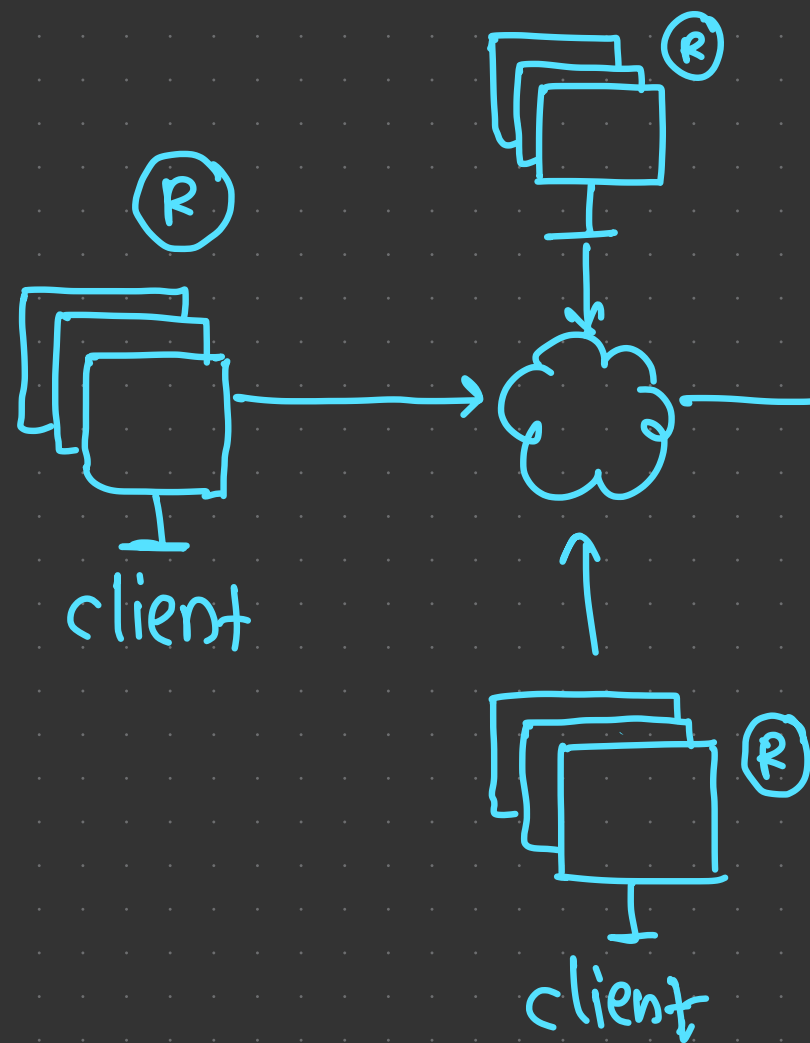
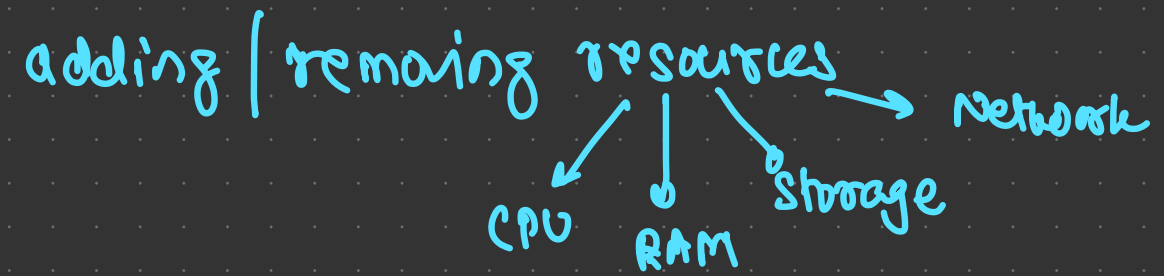
Scalability



- The ability of a computer application or product (hardware or software) to continue to function well when it (or its context) is changed in size or volume in order to meet a user need. Typically, the rescaling is to a larger size or volume
- The rescaling can be of the product itself (for example, a line of computer systems of different sizes in terms of storage, RAM, and so forth) or in the scalable object's movement to a new context (for example, a new operating system)
- It is the ability not only to function well in the rescaled situation, but to actually take full advantage of it
- For example, an application program would be scalable if it could be moved from a smaller to a larger operating system and take full advantage of the larger operating system in terms of performance (user response time and so forth) and the larger number of users that could be handled

- ① static scaling (max no of users)
- ② simple app
- ③ requires less resources
- ④ downtime is considered
- ⑤ tight budget

Vertical Scaling



Thread Pool



server
14.5.6.7
mydomain.com

~~Xeon 2.4 GHz 16T X1~~
~~128 GB RAM~~

Xeon 3.1 GHz 48T X 4
256 GB RAM

Vertical Scaling

add/remove resources

- Vertical scaling (or "scaling up") refers to adding more hardware to an existing machine so that you run the same workload on better specs
- For example, if a server requires more processing power, vertically scaling the device would mean upgrading its CPU. Other examples of vertical scaling include:
 - Adding more RAM to a machine
 - Upgrading a hard disk drive to an SSD with more storage
 - Improving network speed
- This type of scaling was the go-to option back when we had monolithic apps that had to run on a single server
- Nowadays, vertical scaling is a common choice for small-to-mid-size companies that see a noticeable difference in performance or UX from an upgraded machine

↪ single

Vertical Scaling



■ Advantages

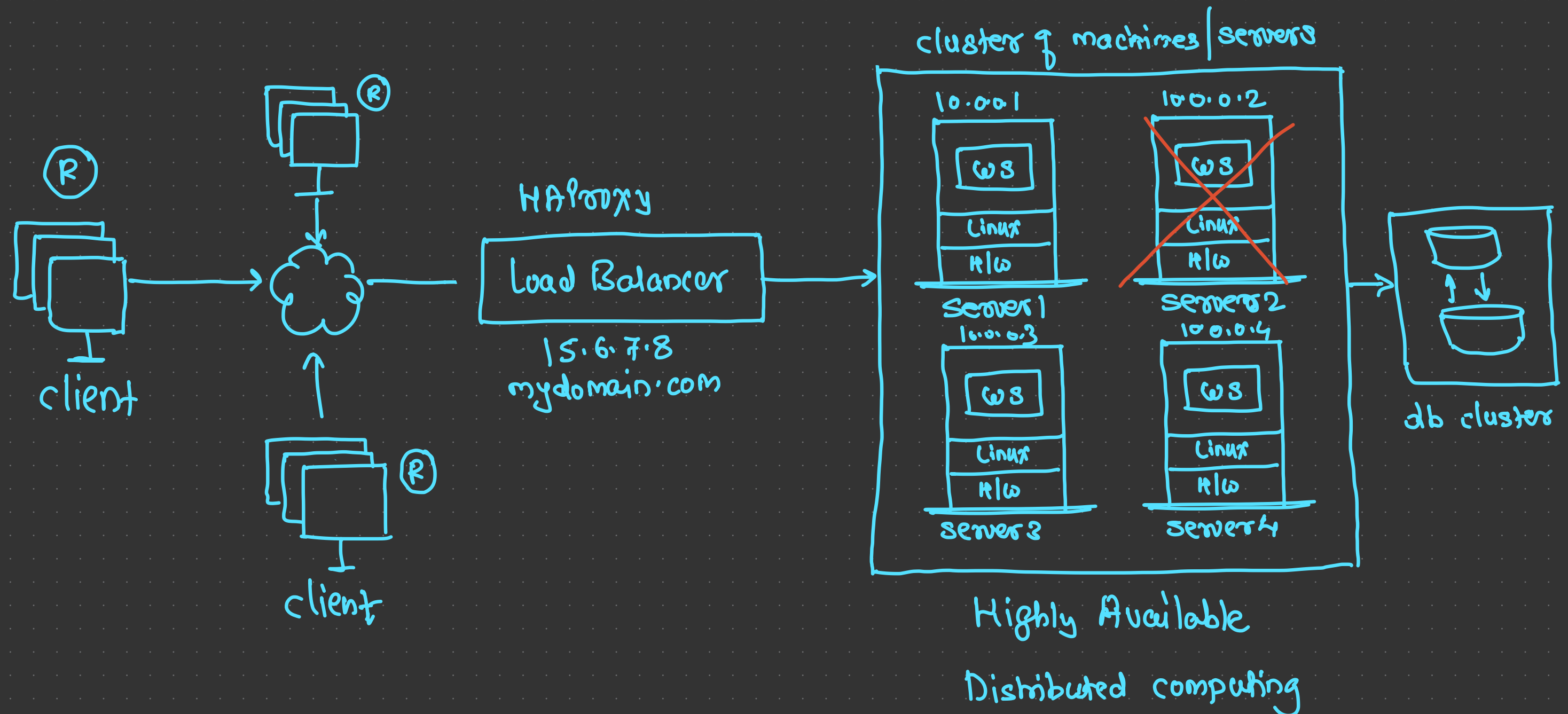
- Fast response times as there are no synchronized nodes that communicate with each other
- Upgrading specific components costs less than purchasing a new server
- You need less space in a server room to scale vertically than horizontally (cooling and power bills are also lower).
- Everything runs on a single node, which leads to better data consistency and integrity
- Easy to implement as there is no need for load balancing or virtualization
- You do not add any extra complexity to day-to-day IT management

■ Disadvantages

- There's a limit to how much a single machine can scale since each device has a threshold for RAM, storage, and processing power
- Higher possibility for service downtime as you rely on one machine that acts as a single point of failure (spot)
- You must take the machine offline to perform the upgrade → downtime
- There is always a percentage of computing resources you are not using actively
- Increased risk of permanent data loss due to hosting everything on a single servers

- ① huge & dynamic scaling
- ② complex application
- ③ microservice application
- ④ Budget
- ⑤ No downtime

Horizontal Scaling





Horizontal Scaling

→ machines (part of cluster)

- Horizontal scaling (or "scaling out") refers to adding more nodes to a system's resource pool to meet the increased demand.
- You do not improve the specifications of the existing machine—instead, you add more same-size servers to the cluster and share the workload across more devices
- Scaling out enables a sysadmin to combine the power of multiple machines to meet the current load demands
- Nodes do not have to be in the same data center, they do not have to be in the same region even

cluster → group of machines known as nodes

Horizontal Scaling



■ Advantages

- Systems that scale out have no scalability cap. The only limits to how much you scale horizontally are the budget and data center space
- Distributing data and operations among nodes increases fault tolerance. If one of the servers goes down, the rest of the system keeps services online and maintains availability
- The system does not go offline during the scaling process
- Systems that scale out also enable you to reduce capacity seamlessly, so you're not stuck always paying for peak demand
- A skilled team knows how to set up a system that scales out automatically (in both ways)
- Sysadmins get to distribute data backups across several nodes

■ Disadvantages

- You must add software for load balancing and ensure nodes synchronize effectively. This setup is challenging for less experienced teams
- Day-to-day server management gets more complex the more nodes you add to a system
- Data consistency is a concern due to cross-server communication
- Node-to-node communication slows down performance in a poorly optimized cluster
- Servers still encounter hardware limits if machines are too small
- Each new server requires more space, cooling, and power, so the costs of scaling out quickly spiral out of control
- More nodes create a larger attack surface and lead to more complex incident response planning



What is load balancing ?

- Load balancing is the method of distributing network traffic equally across a pool of resources that support an application
- Modern applications must process millions of users simultaneously and return the correct text, videos, images, and other data to each user in a fast and reliable manner
- To handle such high volumes of traffic, most applications have many resource servers with duplicate data between them
- A load balancer is a device that sits between the user and the server group and acts as an invisible facilitator, ensuring that all resource servers are used equally

Benefits



- **Application availability**

- Server failure or maintenance can increase application downtime, making your application unavailable to visitors
- Load balancers increase the fault tolerance of your systems by automatically detecting server problems and redirecting client traffic to available servers
- You can use load balancing to make these tasks easier:
 - Run application server maintenance or upgrades without application downtime
 - Provide automatic disaster recovery to backup sites
 - Perform health checks and prevent issues that can cause downtime

- **Application scalability**

- You can use load balancers to direct network traffic intelligently among multiple servers
- Your applications can handle thousands of client requests because load balancing does the following
 - Prevents traffic bottlenecks at any one server
 - Predicts application traffic so that you can add or remove different servers, if needed
 - Adds redundancy to your system so that you can scale with confidence

Benefits



- **Application security**
 - Load balancers come with built-in security features to add another layer of security to your internet applications
 - They are a useful tool to deal with distributed denial of service attacks, in which attackers flood an application server with millions of concurrent requests that cause server failure. Load balancers can also do the following:
 - Monitor traffic and block malicious content
 - Automatically redirect attack traffic to multiple backend servers to minimize impact
 - Route traffic through a group of network firewalls for additional security
- **Application performance**
 - Load balancers improve application performance by increasing response time and reducing network latency
 - They perform several critical tasks such as the following:
 - Distribute the load evenly between servers to improve application performance
 - Redirect client requests to a geographically closer server to reduce latency
 - Ensure the reliability and performance of physical and virtual computing resources



Load Balancing algorithms – Static Load Balancing

- Static load balancing algorithms follow fixed rules and are independent of the current server state. The following are examples of static load balancing
- **Round-robin method**
 - Servers have IP addresses that tell the client where to send requests
 - The IP address is a long number that is difficult to remember
 - To make it easy, a Domain Name System maps website names to servers
 - In the round-robin method, an authoritative name server does the load balancing instead of specialized hardware or software. The name server returns the IP addresses of different servers in the server farm turn by turn or in a round-robin fashion
- **Weighted round-robin method**
 - In weighted round-robin load balancing, you can assign different weights to each server based on their priority or capacity
 - Servers with higher weights will receive more incoming application traffic from the name server
- **IP hash method**
 - In the IP hash method, the load balancer performs a mathematical computation, called hashing, on the client IP address. It converts the client IP address to a number, which is then mapped to individual servers.



Load Balancing algorithms – Dynamic Load Balancing

- Dynamic load balancing algorithms examine the current state of the servers before distributing traffic. The following are some examples of dynamic load balancing algorithms.
- **Least connection method**
 - A connection is an open communication channel between a client and a server
 - When the client sends the first request to the server, they authenticate and establish an active connection between each other
 - In the least connection method, the load balancer checks which servers have the fewest active connections and sends traffic to those servers
 - This method assumes that all connections require equal processing power for all servers.
- **Weighted least connection method**
 - Weighted least connection algorithms assume that some servers can handle more active connections than others
 - Therefore, you can assign different weights or capacities to each server, and the load balancer sends the new client requests to the server with the least connections by capacity
- **Least response time method**
 - The response time is the total time that the server takes to process the incoming requests and send a response
 - The least response time method combines the server response time and the active connections to determine the best server
 - Load balancers use this algorithm to ensure faster service for all users
- **Resource-based method**
 - In the resource-based method, load balancers distribute traffic by analyzing the current server load
 - Specialized software called an agent runs on each server and calculates usage of server resources, such as its computing capacity and memory. Then, the load balancer checks the agent for sufficient free resources before distributing traffic to that server.

Types of load balancing



- **Application load balancing**

- Complex modern applications have several server farms with multiple servers dedicated to a single application function. Application load balancers look at the request content, such as HTTP headers or SSL session IDs, to redirect traffic.
- For example, an ecommerce application has a product directory, shopping cart, and checkout functions. The application load balancer sends requests for browsing products to servers that contain images and videos but do not need to maintain open connections. By comparison, it sends shopping cart requests to servers that can maintain many client connections and save cart data for a long time.

- **Network load balancing**

- Network load balancers examine IP addresses and other network information to redirect traffic optimally
- They track the source of the application traffic and can assign a static IP address to several servers
- Network load balancers use the static and dynamic load balancing algorithms described earlier to balance server load.

- **Global server load balancing**

- Global server load balancing occurs across several geographically distributed servers
- For example, companies can have servers in multiple data centers, in different countries, and in third-party cloud providers around the globe. In this case, local load balancers manage the application load within a region or zone. They attempt to redirect traffic to a server destination that is geographically closer to the client. They might redirect traffic to servers outside the client's geographic zone only in case of server failure

- **DNS load balancing**

- In DNS load balancing, you configure your domain to route network requests across a pool of resources on your domain. A domain can correspond to a website, a mail system, a print server, or another service that is made accessible through the internet
- DNS load balancing is helpful for maintaining application availability and balancing network traffic across a globally distributed pool of resources



- HAProxy (High Availability Proxy) is a high-performance, open-source software load balancer and reverse proxy for TCP and HTTP-based applications
- It is widely regarded as the world's most popular open-source load balancer and is trusted by leading platforms to power their high-traffic services
- **Features**
 - **Load Balancing:** Distributes requests among multiple backend servers using over 13 algorithms including round-robin, least connections, and consistent hashing
 - **High Availability:** Automatically detects failed servers and redistributes traffic to healthy ones
 - **SSL/TLS Termination:** Handles encryption/decryption at the load balancer, reducing backend server load
 - **Health Checks:** Monitors backend server health and removes failed instances automatically
 - **Layer 4 & Layer 7 Support:** Can operate at TCP (transport) level or HTTP (application) level
 - **Rate Limiting:** Controls request rates to protect backend servers from abuse
 - **Observability:** Provides detailed metrics, logs, and a statistics page for monitoring

Architecture



- HAProxy operates on a proxy model with two main components
- **Frontend**
 - Defines IP addresses and ports that accept incoming connections
- **Backend**
 - Defines pools of servers that handle the actual requests
- Additionally, configuration includes:
 - **Global section:** Process-wide settings
 - **Defaults section:** Default parameters inherited by frontends and backends

Configuration



- **# Install HAProxy package**
- `sudo yum install haproxy -y`
- **# Check HAProxy version**
- `haproxy --version`
- **# Check service status**
- `sudo systemctl status haproxy`
- **# configuration file**
- The main configuration file is typically located at `/etc/haproxy/haproxy.cfg`

- **# Backend: Distribute to two web servers**

```
backend http_backend
```

```
    mode http
```

```
    balance roundrobin
```

```
    server web_server1 192.168.1.10:80 check inter 5s rise 2  
    fall 3
```

```
    server web_server2 192.168.1.11:80 check inter 5s rise 2  
    fall 3
```